Aim : Implement Different Types of Joins in SQL

A **JOIN** is used to combine rows from two or more tables based on a related column.

1. Create Tables

2. Insert Values

```
mysql> USE vaishnavi
Database changed
mysql> CREATE TABLE Customers (
    ->     CustomerID INT,
    ->     CustomerName VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Customers (CustomerID, CustomerName) VALUES
    -> (1, 'David'),
    -> (2, 'Emma'),
    -> (3, 'Frank');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE Orders (
    ->     OrderID INT,
    ->     OrderAmount DECIMAL(10,2),
    ->     CustomerID INT
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Orders (OrderID, OrderAmount, CustomerID) VALUES
    -> (9001, 150.00, 1),
    -> (9002, 45.00, 2),
    -> (9003, 85.50, 4);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

1. INNER JOIN

An **INNER JOIN** returns only the rows that have matching values in both tables.

```
mysql> SELECT Customers.CustomerName, Orders.OrderID, Orders.OrderAmount
    -> FROM Customers
    -> INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
+--------------+---------+-------------+
| CustomerName | OrderID | OrderAmount |
+--------------+---------+-------------+
| David        |    9001 |      150.00 |
| Emma         |    9002 |       45.00 |
+--------------+---------+-------------+
2 rows in set (0.00 sec)
```

2. LEFT OUTER JOIN

A **LEFT JOIN** returns **all rows from the left table** and matching rows from the right table.

```
mysql> SELECT Customers.CustomerName, Orders.OrderID, Orders.OrderAmount
    -> FROM Customers
    -> LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
+--------------+---------+-------------+
| CustomerName | OrderID | OrderAmount |
+--------------+---------+-------------+
| David        |    9001 |      150.00 |
| Emma         |    9002 |       45.00 |
| Frank        |    NULL |        NULL |
+--------------+---------+-------------+
3 rows in set (0.00 sec)
```

3. RIGHT OUTER JOIN

A **RIGHT JOIN** returns **all rows from the right table** and matching rows from the left table.

```
mysql> SELECT Customers.CustomerName, Orders.OrderID, Orders.OrderAmount
    -> FROM Customers
    -> RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
+--------------+---------+-------------+
| CustomerName | OrderID | OrderAmount |
+--------------+---------+-------------+
| David        |    9001 |      150.00 |
| Emma         |    9002 |       45.00 |
| NULL         |    9003 |       85.50 |
+--------------+---------+-------------+
3 rows in set (0.00 sec)
```

4. FULL OUTER JOIN

A **FULL OUTER JOIN** returns **all rows from both tables**. Matching rows are combined, and non-matching rows contain NULL.

```
mysql> SELECT Customers.CustomerName, Orders.OrderID, Orders.OrderAmount
    -> FROM Customers
    -> LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID
    ->
    -> UNION
    ->
    -> SELECT Customers.CustomerName, Orders.OrderID, Orders.OrderAmount
    -> FROM Customers
    -> RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
+--------------+---------+-------------+
| CustomerName | OrderID | OrderAmount |
+--------------+---------+-------------+
| David        |    9001 |      150.00 |
| Emma         |    9002 |       45.00 |
| Frank        |    NULL |        NULL |
| NULL         |    9003 |       85.50 |
+--------------+---------+-------------+
4 rows in set (0.00 sec)
```

Note: MySQL does not directly support FULL OUTER JOIN; it can be implemented using LEFT JOIN, RIGHT JOIN, and UNION.

5. CROSS JOIN

A **CROSS JOIN** produces the **Cartesian product** of two tables. It combines **each row of the first table with every row of the second table**.

SELECT Student.Student_Name, Course.Course_Name

FROM Student

CROSS JOIN Course;

If there are **4 students** and **4 courses**, the result will contain:

4 × 4 = 16 rows

Example output:

Student_Name	**Course_Name**
Rahul	BCA
Rahul	BBA

Student_Name	Course_Name
Rahul	BCom
Rahul	MCA
Priya	BCA
Priya	BBA
...	...

👉 **No ON condition is required** for a CROSS JOIN.

6. SELF JOIN

A **SELF JOIN** means joining a table **with itself**.

For example, create an Employee table:

```
CREATE TABLE Employee (
    Employee_ID INT,
    Employee_Name VARCHAR(50),
    Manager_ID INT
);
```

Insert values:

```
INSERT INTO Employee VALUES
(101, 'John', NULL),
(102, 'Rahul', 101),
(103, 'Priya', 101),
(104, 'Amit', 102);
```

Here:

- John → Manager
- Rahul → John's employee

- Priya → John's employee
- Amit → Rahul's employee

Query

```
SELECT
    E.Employee_Name AS Employee,
    M.Employee_Name AS Manager
FROM Employee E
LEFT JOIN Employee M
ON E.Manager_ID = M.Employee_ID;
```

Output

Employee	Manager
John	NULL
Rahul	John
Priya	John
Amit	Rahul

Here, Employee is used **twice**:

- E represents the employee.
- M represents the manager.

Final list of joins

1. **INNER JOIN** → Only matching rows
2. **LEFT OUTER JOIN** → All left table rows + matching right rows
3. **RIGHT OUTER JOIN** → All right table rows + matching left rows
4. **FULL OUTER JOIN** → All rows from both tables
5. **CROSS JOIN** → Every possible combination of rows

6. **SELF JOIN** → A table joined with itself

Conclusion: Joins are used to retrieve/select related data from multiple tables efficiently based on common columns.